

Capstone Project: Enterprise Operations Assistant

1. Business Scenario

A large organization operates a centralized customer operations team that handles customer complaints related to billing, payments, refunds, and account discrepancies.

Operations executives currently spend significant time investigating complaints across multiple systems. They may need to retrieve customer information from an internal database, verify billing information with an external service, reconcile discrepancies, determine an appropriate resolution, and document the outcome.

The organization wants to build an **Enterprise Operations Assistant** that can automate and coordinate these activities while ensuring that sensitive financial actions remain under human supervision.

2. Project Objective

Build an end-to-end **multi-agent operations assistant** that can receive a customer complaint, investigate the issue using internal and external data sources, reconcile discrepancies, recommend an appropriate resolution, and securely record the outcome.

The assistant should be capable of dynamically determining which operations are required rather than following a fixed sequence for every complaint.

Financial adjustments must require explicit approval from an authorized supervisor before they are executed.

3. Example Business Request

An operations executive submits the following request:

"Customer C1024 has complained that they were charged ₹12,500 for their latest invoice, but the amount shown by the billing system is ₹10,000. Investigate the discrepancy and recommend an appropriate resolution."

The system should investigate the complaint by retrieving the relevant customer and invoice information, checking the external billing system, reconciling the two sets of information, and determining the appropriate resolution.

If the resolution requires a financial adjustment, the workflow must pause and request supervisor approval before making the adjustment.

4. Functional Requirements

A. Complaint Analysis

The system should accept complaints in natural language.

Example complaints include:

- Incorrect invoice amount
- Duplicate billing
- Payment recorded incorrectly
- Refund not received
- Incorrect account charge
- Billing amount mismatch

The system should identify the nature of the complaint and determine which information and operations are required.

B. Customer and Billing Database

Create a SQLite database containing sample operational data.

Suggested tables:

Customers

Field	Example
customer_id	C1024
name	Rahul Sharma
email	rahul@example.com
account_status	Active

Invoices

Field	Example
invoice_id	INV10045
customer_id	C1024
invoice_amount	12500
invoice_status	Paid
invoice_date	2026-09-20

Adjustments

Field	Example
adjustment_id	ADJ1001
customer_id	C1024

Field	Example
invoice_id	INV10045
amount	2500
status	Pending
reason	Billing discrepancy

The database should support both **read and write operations**.

5. Mock Third-Party Billing API

Create or use a mock REST API representing an external billing system.

For example:

```
GET /billing/{invoice_id}
```

Example response:

```
{
  "invoice_id": "INV10045",
  "customer_id": "C1024",
  "amount": 10000,
  "currency": "INR",
  "status": "PAID",
  "billing_date": "2026-09-20"
}
```

The assistant should access the API through a tool.

The external billing information should be compared with the internal database information to identify discrepancies.

6. Multi-Agent Architecture

The solution must contain **at least three specialized agents** coordinated through a LangGraph-based workflow.

6.1 Complaint Investigation Agent

Responsible for:

- Understanding the complaint
 - Identifying the customer and relevant invoice
 - Retrieving customer information
 - Retrieving internal billing information
 - Calling the external billing API when required
-

6.2 Reconciliation Agent

Responsible for:

- Comparing internal and external billing information
- Identifying discrepancies
- Determining the financial impact
- Producing a structured reconciliation result

For example:

```
Internal Invoice Amount : ₹12,500
External Billing Amount : ₹10,000
Difference               : ₹2,500
Status                  : Discrepancy Detected
```

6.3 Resolution Agent

Responsible for:

- Reviewing the investigation results
 - Reviewing the reconciliation results
 - Determining an appropriate resolution
 - Preparing a resolution recommendation
 - Determining whether a financial adjustment is required
-

6.4 Supervisor Agent

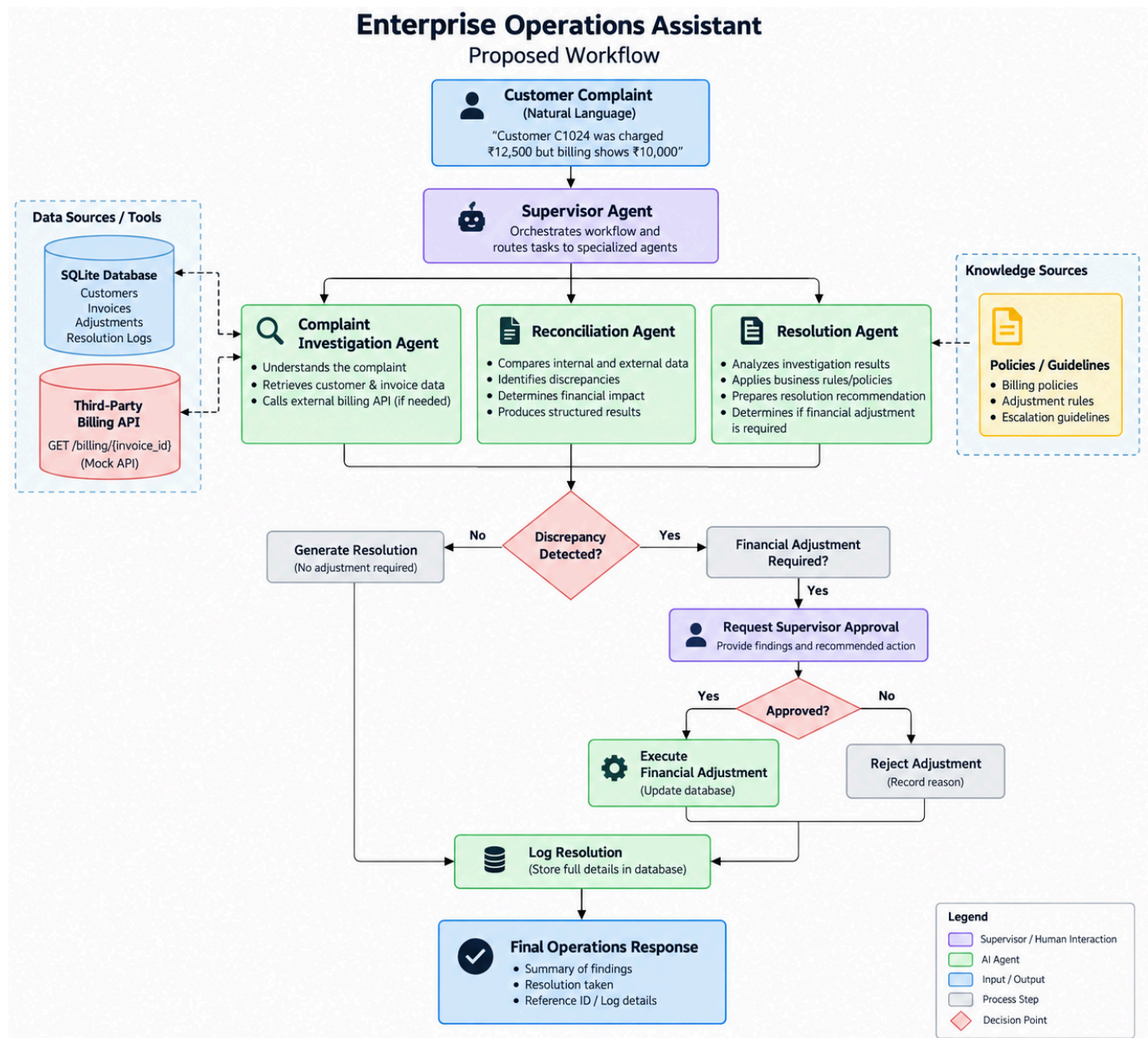
A supervisor agent should coordinate the specialized agents and control the overall workflow.

The supervisor should determine:

- Which agent should execute next
 - Whether additional information is required
 - Whether the issue has been sufficiently investigated
 - Whether a financial adjustment is required
 - Whether human approval is necessary
 - When the workflow can be completed
-

7. Proposed Workflow

The overall workflow may follow the structure below.



The exact graph structure may differ, provided the implementation demonstrates meaningful agent coordination and state-based workflow management.

8. Required Tools

Participants should expose appropriate enterprise operations as tools.

Suggested tools include:

```
get_customer(customer_id)
```

```
get_invoice(invoice_id)

get_customer_invoices(customer_id)

get_external_billing(invoice_id)

reconcile_invoice(invoice_id)

create_financial_adjustment(
    customer_id,
    invoice_id,
    amount,
    reason
)

update_adjustment_status(
    adjustment_id,
    status
)

log_resolution(
    customer_id,
    complaint,
    resolution
)
```

Tools should have well-defined input schemas and should return structured results wherever practical.

9. Human-in-the-Loop Requirement

Financial adjustments must **not be executed automatically**.

For example, if the system determines that the customer should receive a ₹2,500 adjustment, the workflow should pause before executing the financial operation.

The supervisor should be presented with information such as:

```
Customer      : C1024
Invoice       : INV10045
Internal Amount: ₹12,500
External Amount: ₹10,000
Difference    : ₹2,500

Finding:
Billing discrepancy detected.

Recommended Action:
Create a ₹2,500 financial adjustment.
```

Reason:
Internal invoice amount differs from the verified third-party billing amount.

The supervisor can then:

APPROVE
REJECT

The workflow should resume based on the decision.

10. Secure Resolution Logging

Every completed complaint should be recorded in the database.

The log should capture information such as:

- Customer ID
- Complaint
- Invoice ID
- Investigation result
- Reconciliation result
- Recommended resolution
- Financial adjustment amount
- Approval decision
- Final status
- Timestamp

The purpose is to provide an auditable record of how the complaint was handled.

11. Observability Requirements

The complete workflow should be instrumented using **LangSmith** or **Langfuse**.

Participants should be able to demonstrate traces covering:

- User request
- Supervisor decisions
- Agent execution
- LLM calls
- Tool calls
- Tool inputs and outputs
- State transitions
- Human approval

- Final response

The objective is to provide **100% trace coverage** for the implemented workflow.

Participants should be able to use the traces to identify issues such as:

- Incorrect agent selection
- Incorrect tool invocation
- Invalid tool parameters
- Unexpected state transitions
- Excessive latency
- Incorrect final responses

12. Evaluation Requirements

Create an automated evaluation suite containing at least **five representative scenarios**.

Suggested scenarios:

Scenario	Expected Behavior
Billing amount mismatch	Detect and reconcile the discrepancy
Duplicate charge	Identify duplicate billing and recommend resolution
Correct billing	Confirm that no adjustment is required
Financial adjustment	Pause and request human approval
Invalid customer/invoice	Handle the error gracefully

The evaluation should capture at least:

Accuracy

Determine whether the system:

- Identified the correct issue
- Retrieved the correct information
- Selected the appropriate tools
- Produced the expected resolution
- Applied human approval correctly

Latency

Measure the execution time for each scenario.

A simple evaluation report could be:

Scenario	Result	Accuracy	Latency
Billing mismatch	Pass	100%	4.2 sec
Duplicate charge	Pass	100%	5.1 sec
Correct billing	Pass	100%	3.8 sec
Financial adjustment	Pass	100%	5.7 sec
Invalid invoice	Pass	100%	2.9 sec

The actual values should be generated from the participant's implementation rather than hard-coded.

13. Expected Final Response

After completing the investigation, the assistant should provide a concise operational summary.

For example:

```
Complaint Investigation Completed
```

```
Customer: C1024
```

```
Invoice: INV10045
```

```
Finding:
```

```
The internal billing system shows an amount of ₹12,500, while the  
third-party billing system reports ₹10,000.
```

```
Discrepancy:
```

```
₹2,500
```

```
Recommended Resolution:
```

```
Create a ₹2,500 financial adjustment for the customer.
```

```
Approval:
```

```
Supervisor approval required.
```

```
Status:
```

```
Pending supervisor approval.
```

After approval, the response should reflect the resulting status and the financial adjustment.

14. Technical Expectations

The implementation should demonstrate the following capabilities:

- Python

- LangChain
- Structured tool calling
- SQLite integration
- REST API integration
- RAG or retrieval-based access where appropriate
- LangGraph
- Multi-agent orchestration
- Supervisor pattern
- Stateful workflow
- Conditional routing
- Human-in-the-loop
- Database read/write operations
- Error handling
- LangSmith or Langfuse tracing
- Automated evaluation

The solution should be implemented as a functional prototype rather than a production banking/financial system.

15. Deliverables

Participants should provide:

1. Python source code
2. SQLite database containing sample data
3. Mock third-party billing API
4. Sample complaint scenarios
5. LangGraph workflow
6. Specialized agents and tools
7. Human-approval workflow
8. LangSmith/Langfuse traces
9. Automated evaluation script
10. Evaluation results covering the five scenarios
11. README containing:
 - Solution architecture
 - Setup instructions
 - Agent responsibilities
 - Tools implemented
 - Sample requests
 - Sample responses
 - Evaluation results